

Level 2 Documentation: 3D Maze Mapping with Iris Drone, ArduPilot SITL, Gazebo Harmonic, and ROS 2

1. Overview & Objective

The objective of Level 2 is to deploy an autonomous Iris quadcopter equipped with a 3D LiDAR sensor inside a complex 3D maze environment using Gazebo Harmonic. The pipeline integrates **ArduPilot SITL** for flight control, **ROS 2 Humble** for middleware communication, and **Cartographer SLAM** for real-time 3D mapping of the environment.

2. System Architecture & Tech Stack

- **Operating System:** Windows Subsystem for Linux (WSL - Ubuntu 22.04 LTS)
- **Middleware:** ROS 2 Humble Hawksbill
- **Simulation Engine:** Gazebo Harmonic (Sim/Ignition) with OGRE Rendering Engine
- **Flight Controller:** ArduPilot SITL (ArduCopter) + MAVProxy
- **Sensor Payload:** 3D LiDAR (`gpu_lidar` with 360° horizontal sweep and 60 vertical channels)
- **Mapping Framework:** ROS 2 Cartographer / Slam Toolbox

3. Step-by-Step Implementation Guide

Step 1: Environment Preparation & Workspace Build

1. Clone and configure the required packages (`ardupilot_gz`, `ros_gz` bridge, and descriptions) inside your ROS 2 workspace (`~/ros2_ws`).
2. Build the workspace while managing overlay/underlay dependencies and allowing package overrides for Gazebo integration:

•
Shell

```
cd ~/ros2_ws
colcon build --symlink-install --packages-select
ardupilot_gz_description ardupilot_gz_bringup
source install/setup.bash
```

Step 2: Launching the Gazebo Maze World (Terminal 1)

Initialize the Gazebo simulation server and load the custom maze world containing the Iris drone with the 3D LiDAR sensor:

-

Shell

```
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
export GZ_VERSION=harmonic
export GZ_RENDERING_ENGINE=ogre
ros2 launch ardupilot_gz_bringup soham_maze.launch.py
```

Step 3: Running ArduPilot SITL (Terminal 2)

Bypass wrapper timing jitter by launching the ArduPilot SITL binary directly with the Gazebo-Iris model configuration:

-

Shell

```
cd ~/ardupilot
./build/sitl/bin/arducopter --model gazebo-iris --speedup 1
--sim-address=127.0.0.1 -I0
```

Step 4: Connecting MAVProxy and Commanding Flight (Terminal 3)

Connect MAVProxy to the SITL instance via TCP, establish guided mode, arm the drone, and execute a vertical takeoff:

-

Shell

```
mavproxy.py --master=tcp:127.0.0.1:5760 --out 172.28.48.1:14550
--map --console
```

Inside the MAVProxy console, execute the flight sequence:

-

None

```
mode guided
arm throttle
takeoff 2
```

Step 5: Launching Cartographer SLAM (Terminal 4)

Verify that the 3D LiDAR topic (`/lidar`) is actively publishing point cloud/laser scan data, then launch the mapping node:

-

Shell

```
cd ~/ros2_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
export GZ_VERSION=harmonic
ros2 launch ardupilot_cartographer cartographer.launch.py
```

4. Key Troubleshooting & Optimizations Handled

- **Timing & Clock Jitter:** Addressed WSL virtualization lag by removing strict lockstep requirements (`<lock_step>0</lock_step>`) and utilizing direct binary execution with explicit speedup configuration (`--speedup 1`).
- **Sensor Integration:** Configured a custom `lidar_3d` model featuring a 60-row vertical scan matrix to capture high-density vertical profiles of the maze walls.
- **Port Synchronization:** Ensured seamless socket binding between Gazebo's `ArduPilotPlugin` and ArduPilot SITL to eliminate protocol magic and heartbeat timeout errors.

Map:



